VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Quiz and Question Bank Management System

A9601 · Object Oriented Programming

Submitted by

Batch No. 19

V.Ganesh - 25881A05CR
Palle Akanksha - 25881A05CA
T.Manikanta - 25881A05DF

Course Facilitator

Mr. Ganesh Deshmukh

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

Contents

1. Introduction..... 3

2. Literature Review 3

3. Objectives 3

4. Methodology..... 4

4.1 System Architecture / Block Diagram..... 4

4.2 System Design and Components..... 4

4.3 Working Principle 5

4.4 Software Implementation..... 6

5. Results and Discussion 7

6. Mapping POs and SDGs..... 8

7. Conclusion and Future Scope..... 9

Bibliography..... 9

Abstract

Managing quiz questions manually can make it difficult to organize questions by subject or topic, prepare quizzes consistently, and keep track of scores. This project proposes a **Quiz and Question Bank Management System**, a Java desktop application built with Java Swing and a MySQL database accessed through JDBC. It provides a structured way to add, view, update, delete, search, and categorize questions, and to create a quiz from the stored question bank.

The design applies core Object-Oriented Programming concepts through separate model, database-access, and graphical-interface classes. Swing components provide the user interface, event listeners respond to button actions, collections manage question records in memory, JDBC performs persistent database operations, and input validation and exception handling improve reliability. File handling can be used to export/import question sets, while serialization can provide a local backup of question objects. The system can be extended with timed quizzes, user accounts, randomized questions, and detailed performance reports.

Keywords: Java Swing, Question Bank, Quiz Management, JDBC, MySQL, CRUD, Event Handling, Collections, File Handling.

1. Introduction

Quizzes are commonly used to check understanding, support revision, and assess learning. When questions are stored in separate documents or notebooks, it can be difficult to find a particular question, avoid duplication, maintain answer keys, or prepare quizzes for different topics. A centralized question bank can make these activities easier by storing question text, options, the correct answer, subject, difficulty level, and marks in a consistent format.

This project describes a desktop application developed using Java Swing and MySQL. The interface allows a user to manage questions through forms and tables. JDBC connects the Java application to the database, and SQL operations keep question records available after the application closes. The project demonstrates object-oriented design, graphical user interface development, event handling, database connectivity, collections, file handling, validation, and exception handling.

2. Literature Review

Question banks are often maintained in spreadsheets, word-processing files, learning-management systems, or dedicated quiz platforms. Documents and spreadsheets are easy to start with, but they can become difficult to maintain as the number of questions increases. Dedicated online platforms may provide advanced features, but they may require an account, network access, or a larger system than is needed for a small course project.

A local Java desktop application offers a manageable alternative for demonstrating fundamental software engineering concepts. Java Swing provides standard interface components and event-driven interaction [1]. JDBC provides a standard API for connecting Java programs to relational databases [2], and MySQL Connector/J acts as the JDBC driver for MySQL [3]. Parameterized SQL statements are used in the design to handle user-entered values safely and reduce SQL-injection risk.

3. Objectives

- To design a desktop application for maintaining a reusable question bank.
- To store question details persistently in MySQL using JDBC.
- To implement Create, Retrieve, Update, and Delete (CRUD) operations for questions.
- To search and filter questions by subject, topic, or difficulty.
- To create a quiz from stored questions and calculate the user's score.
- To apply Java Swing event handling, collections, file handling, and object-oriented design.
- To validate input and show clear error messages when invalid data or database errors occur.

4. Methodology

4.1 System Architecture / Block Diagram

The system is organized into layers so that the user interface, application logic, and database operations are separated. The GUI collects input and displays results; the data-access class executes SQL; and the database stores question records.

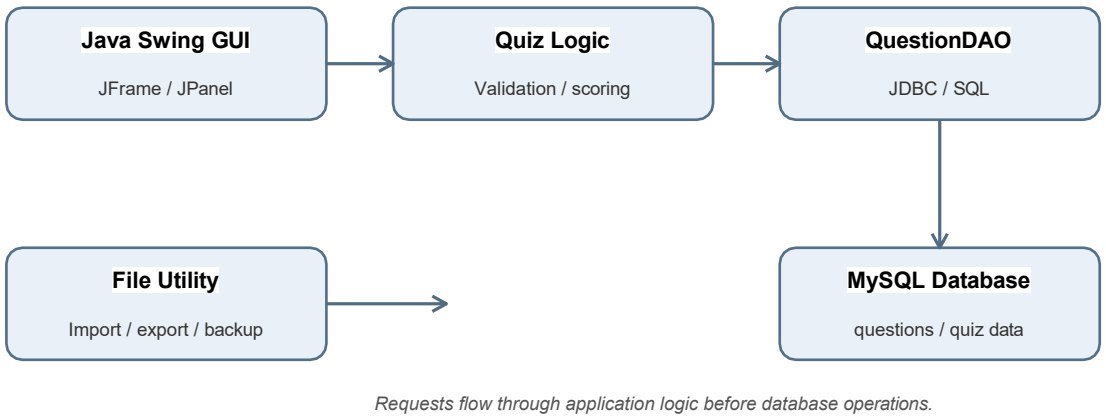


Figure 1: Proposed architecture of the Quiz and Question Bank Management System

The GUI does not execute SQL directly. Instead, it calls the question data-access component, which communicates with MySQL. This separation makes the system easier to maintain and allows the interface and database logic to be tested independently.

4.2 System Design and Components

Components used

• Java Development Kit (JDK) and Swing library
• MySQL Server and a question-bank table
• MySQL Connector/J JDBC driver
• Swing components such as JFrame, JPanel, JLabel, JTextField, JTextArea, JComboBox, JButton, JTable, and JOptionPane
• Java collections such as ArrayList and HashMap where suitable
• File I/O classes for importing and exporting question data

Database design. A main table named *questions* can contain an auto-incremented question ID, question text, subject, topic, difficulty, four answer options, correct option, and marks. A separate quiz-attempt table may store quiz date, total questions, score, and percentage if attempt history is included in the implemented version.

Field	Purpose	Example / rule
question_id	Unique identifier	Auto-increment primary key
question_text	Question statement	Required text
subject / topic	Classification	E.g., Java / OOP
option_a … option_d	Possible answers	Options should be provided
correct_option	Answer key	A, B, C, or D
difficulty / marks	Quiz settings	Difficulty label / positive marks

Table 1: Suggested structure of the questions table

User interface. The main window can contain a question-entry form, a question table, search/filter controls, and buttons for Add, Update, Delete, Clear, and Create Quiz. A quiz screen presents one question at a time or a list of questions, records selected answers, and displays the final score.

4.3 Working Principle

1. The application starts and establishes a connection to the MySQL database.
2. Existing questions are retrieved and displayed in a JTable.
3. The user enters question text, options, answer key, subject, difficulty, and marks.
4. When an action button is clicked, its ActionListener reads and validates the form.
5. If the input is valid, the data-access class executes the corresponding parameterized SQL statement.
6. The question table is refreshed so the displayed records match the database.
7. When a quiz is started, the selected questions are presented, answers are checked against the stored answer key, and the score is calculated.
8. The result is displayed to the user; if attempt history is implemented, the result is stored in the database.

4.4 Software Implementation

Object-oriented design: A Question model class represents one question. A QuestionDAO class contains database methods, DBConnection manages JDBC connections, QuizService handles quiz selection and scoring, and QuizGUI builds the Swing interface. Keeping responsibilities separate avoids putting all program logic in a single class.

CRUD with JDBC: Methods such as addQuestion(), getAllQuestions(), searchQuestions(), updateQuestion(), and deleteQuestion() execute SQL statements using PreparedStatement and try-with-resources. The database connection and statements are closed after use.

Event handling: Buttons use ActionListener to respond to user actions. A table-selection listener can copy the selected question into the form for editing. Quiz option buttons or radio buttons record the selected answer.

Collections: An ArrayList can hold questions retrieved from the database or selected for a quiz. A HashMap can map question IDs to selected answers or store category counts. Collections are used where they simplify in-memory processing.

File handling and backup: FileReader/FileWriter or buffered streams can be used to import or export question sets in CSV format. If Serializable model objects are used, ObjectOutputStream and ObjectInputStream can support local backup and restore. Imported records must be validated before they are saved.

Validation and exceptions: Required text fields must not be empty, the correct answer must match one of the available options, marks must be positive, and database or file errors must be shown as readable messages rather than raw stack traces.

Field	Validation rule	Example message
Question text	Required; must not be blank	Enter the question text.
Options	Required options; avoid accidental blanks	Complete all answer options.
Correct answer	Must be one of A, B, C, or D	Select a valid correct option.
Marks	Numeric value greater than zero	Marks must be positive.
Subject / topic	Must be selected or entered	Choose a subject.

Table 2: Suggested input validation rules

5. Results and Discussion

The expected result is a desktop application that provides a single place to maintain questions and prepare quizzes. When a question is added, updated, or deleted successfully, the table should refresh to show the current stored records. Search and filtering should help the user locate questions by keyword, subject, topic, or difficulty. During a quiz, the application should compare selected answers with the answer key and calculate a score based on the configured marks.

The following table summarizes the intended behaviour. Confirm each item against the features that are actually implemented in your submitted application.

Feature	Action	Expected result
Add question	Complete form and click Add	Question is saved and table refreshed
Update question	Select row, edit, click Update	Stored question is updated
Delete question	Select row and confirm Delete	Question is removed
Search/filter	Enter keyword or select category	Matching questions are displayed
Create quiz	Choose quiz settings and start	Questions are presented to answer
Score calculation	Submit answers	Correct answers and score are shown
Invalid input	Leave required fields blank	Validation message appears; no invalid record saved
Database unavailable	Database connection fails	Clear connection error is shown

Table 3: Feature behaviour summary

Using PreparedStatement for database operations helps handle special characters and reduces SQL-injection risk. Separating the GUI, model, quiz logic, and data-access code improves readability and makes future modifications easier. Possible limitations of a basic version include a single-user design, limited reporting, and the need to manage database setup on the machine where the application runs.

Recommended screenshots to add before submission

- Main screen showing the question-entry form and question table.
- Example of adding or updating a question.
- Search/filter results.
- Quiz screen with answer options.
- Final score/result screen.
- Database table showing saved question records.

6. Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

The project applies programming, database, design, and teamwork skills. The mapping below explains the relevance of selected program outcomes to this project.

PO	Program outcome	Relevance to the project
PO1	Engineering Knowledge	Uses object-oriented programming, Java, SQL, and database concepts.
PO2	Problem Analysis	Identifies issues in manually storing, searching, and organizing questions.
PO3	Design/Development of Solutions	Designs a GUI, question model, quiz logic, and database structure.
PO5	Modern Tool Usage	Uses Java Swing, JDBC, MySQL, and development tools.
PO9	Individual and Team Work	Divides interface, database, testing, and documentation tasks among team members.

Table 4: Mapping of Program Outcomes to project relevance

6.2 Sustainable Development Goals (SDGs) Mapping

The project supports learning and the responsible use of digital information by helping users organize educational questions in a reusable format.

SDG	Goal	Relevance
SDG 4	Quality Education	Supports quiz-based practice and structured question organization for learning.
SDG 9	Industry, Innovation and Infrastructure	Applies software and database technology to an educational task.
SDG 12	Responsible Consumption and Production	Encourages reusable digital question records instead of repeated paper-based lists.

Table 5: Mapping of Sustainable Development Goals to project relevance

7. Conclusion and Future Scope

The Quiz and Question Bank Management System is designed to organize questions in a structured, searchable database and support the preparation and completion of quizzes through a Java Swing interface. The proposed design brings together object-oriented programming, event handling, JDBC connectivity, SQL CRUD operations, collections, file handling, and input validation. It provides a practical example of applying course concepts to an educational use case.

The project can be enhanced by adding separate administrator and student roles, timed quizzes, randomized question selection, negative marking where appropriate, topic-wise performance reports, CSV templates, and a web-based interface. Further improvements could include password-protected accounts and a richer dashboard for tracking quiz attempts.

Bibliography

[1] Oracle, “Creating a GUI With Swing,” The Java Tutorials. Available: https://docs.oracle.com/javase/tutorial/uiswing/

[2] Oracle, “JDBC Basics,” The Java Tutorials. Available: https://docs.oracle.com/javase/tutorial/jdbc/basics/

[3] MySQL, “MySQL Connector/J Developer Guide.” Available: https://dev.mysql.com/doc/connector-j/en/

[4] MySQL, “MySQL Reference Manual.” Available: https://dev.mysql.com/doc/refman/